

Improving Interface Design in Interactive Task Learning for Hierarchical Tasks based on a Qualitative Study

Jieyu Zhou

Georgia Institute of Technology
Atlanta, Georgia, USA

Christopher MacLellan

Georgia Institute of Technology
Atlanta, Georgia, USA

ABSTRACT

Interactive Task Learning (ITL) systems acquire task knowledge from human instructions in natural language interaction. The interaction design of ITL agents for hierarchical tasks stays uncharted. This paper studied Verbal Apprentice Learner (VAL) for gaming, as an ITL example, and qualitatively analyzed the user study data to provide design insights on dialogue language types, task instruction strategies, and error handling. We then proposed an interface design: Editable Hierarchy Knowledge (EHK), as a generic probe for ITL systems for hierarchical tasks.

KEYWORDS

Hierarchical Task, Task Agent

ACM Reference Format:

Jieyu Zhou and Christopher MacLellan. 2024. Improving Interface Design in Interactive Task Learning for Hierarchical Tasks based on a Qualitative Study. In *The 37th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '24)*, October 13–16, 2024, Pittsburgh, PA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3672539.3686326>

1 INTRODUCTION

Task-oriented agents are widely used across various domains [7, 12, 18], including smart speakers (Alexa, Google Home, Siri), multi-model devices (Rabbit R1 [15], AI Pin), web-based systems [5, 9, 10], and game controls [8, 17, 19]. They communicate between different environments, by translating user requests to application programming interfaces (API) for task execution. User's natural language input is often syntactically loose and ambiguous, but the APIs require rigid, specific actionable commands in the programming language. Unlike chatbots, which simply answer the query in an isolated environment, with limited ability to address the challenges of translating loose natural language to rigid API commands, task-oriented agents guarantee the robustness of the translation and the correctness of execution.

Moreover, when the built-in API commands are insufficient for a user's instruction, task-oriented agents should learn from the user to customize the API commands. Interactive Task Learning (ITL) system is one approach to this issue. ITL agents acquire task knowledge, such as task names and their subtasks, from human instructions provided via natural language interaction [4, 7]. Interaction with ITL systems for relatively simple tasks, such as "ordering

Cappuccino on Starbucks website", has been widely explored in HCI field. For example, ONYX conducts data visualization works [16], SOVITE refers to third-party apps to repair conversational breakdowns implementing on flight booking [11], and Sugilite enables users to create automation on Android systems [10]. However, when ITL systems face complicated tasks with multiple steps and hierarchical structures [6, 8, 13], which kinds of interaction are suitable needs to be further explored.

Verbal Apprentice Learner (VAL) [8] is one prior work that explored user interaction for complicated tasks. VAL is a neuro-symbolic ITL system, capable of interactively acquiring task knowledge from natural language instruction provided by end users tested in game environments [8]. It is neuro by leveraging LLMs to address the challenges associated with parsing error-prone natural language inputs, a common issue in prior ITL systems. On the other hand, it is symbolic by engaging in multi-step reasoning through a hierarchical task network (HTN) planning system [2] and grounding a task to default primitive actions (executable API commands). The original VAL study [8] focused primarily on evaluating the accuracy and performance of VAL's components. However, the interaction transcripts and open-ended survey results have never been analyzed. In this paper, we analyzed the data, generating design insights and new VAL 2.0 prototypes. We believe this work shall also provide insights for other ITL systems for complicated hierarchical tasks.

2 DESIGN INSIGHTS

We analyzed the log data of the user study [8], where 12 participants were asked to teach VAL to make onion soup in the Overcooked game. After the study, there was a post-session survey, consisting of seven-point Likert-scale questions and open-ended questions asking users comments based on their experience. We applied a reflexive thematic analysis approach [3] on 1336 pieces of log data paired with participants' survey results.

2.1 Use the type of language desired from users

We found participants used two distinct types of language (some used both): 1. natural language (12 participants used), similar to human communication in daily life. Example: "Could you please go grab an onion and move to the onion storage tile? Do you know where onion storage tile is?" (P1); 2. coding language (5 participants used), has the format "action(object)", such as make(onion, pot), pressSpace(onion). VAL mostly interacts in natural language but also uses coding language, such as "I have learned the action: cook(onion)" in some scenarios. This sometimes leads participants to mirror coding language from VAL. Unfortunately, VAL cannot parse coding language very well, which leads to errors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '24, October 13–16, 2024, Pittsburgh, PA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0718-6/24/10

<https://doi.org/10.1145/3672539.3686326>

In general, task-oriented agents act as a connection between users' inputs in natural language and concise API command output in coding language. The designer shall avoid directly displaying the back-end API commands in the front-end dialog, as users mirror the language used by the agent. Therefore, to avoid mode errors [14], the dialog should keep only using the natural language type.

2.2 Handle different task instruction strategies

Users may apply different strategies to teach the ITL agent to conduct a complicated task hierarchically. A typical task hierarchy of cooking onion soup is shown on the right side of Figure 1. In VAL, We identified three strategies to form the hierarchy: 1. top-down (8 participants used), beginning with the task name and listing all the sub-steps; 2. bottom-up (3 participants used) starting with detailed steps and then combining into a higher-level task, and 3. intermediate (1 participant used) starting with middle-level tasks of cooking onion soup, such as "boiling the onion", "plating the soup" and then grounding them out to primitive actions (press space and move to).

We observed that bottom-up strategies sometimes cause a shallow hierarchy, which only contains one layer (the main task and a sequence of primitive actions under it), especially when the user describes the task steps in a very concise way without any redundant words, such as "Move to onion, press space, move to pot, press space, press space" (P3). We perceive a shallow hierarchy as a learning failure because it compromises the generalizability of the learning task. If a user wants to teach VAL a new task containing subtasks it already knows, a deep hierarchy can reuse some of its intermediate steps, while a flat hierarchy must limits reusability. The design insight drawn from this issue is to ask the users to clarify the higher-level tasks when a flat hierarchy is detected.

2.3 Offer multiple ways for error handling

Error handling is a major requirement for task-oriented agents, they should prevent an error before it happens or recover from it after it occurs. Unlike single-or-few-step tasks, whose error handling has been discussed [10, 11], the complicated multi-step scenario remains uncharted. In multi-step tasks, an error in one step can propagate and eventually compromise executing the whole task. Besides execution errors faced by all other task-oriented agents, ITL agents may encounter extra potential errors in parsing due to its interactive nature (i.e., mistakes in the task names or objects).

One approach is to prevent errors by confirming the user instruction in a dialog. However, frequent confirmation dialog can frustrate the user. This is particularly true for multi-step task ITLs where the dialog can be overwhelming for the user because each parsing contains multiple input confirmations and each task has multiple steps. From VAL's log data, participants experienced an average of 45 confirmations to make onion soup, and they complained: "Although I found it helpful to check that VAL parsed my instructions correctly, it also felt tedious to answer them all the time." (P11). How might we reduce the tedious confirmation without risking the system's correctness? One approach is to compensate by providing easy recoveries after any error occurs. Balancing error prevention and recovery remains critical to designing a robust ITL system.

VAL has a simple recovery strategy: an option to undo instructions which was used by 3/4 participants in the tests. However, a

straightforward undo is not enough, because users do not often notice their mistakes immediately and continue making commands. After noticing an error, they must also undo the sequential correct instructions. As one participant said: "It was difficult to undo or go back. I first told VAL an incorrect sequence, and it was not easy to re-teach VAL from scratch." (P12). Designers of ITL systems for multi-step tasks should offer multiple error-handling strategies that account for the interrelationship of steps and the balance between prevention and recovery to avoid tediousness.

3 DESIGN PROTOTYPES

Based on the design insights, we propose VAL 2.0 which fetures a revised the dialog interface and a new Graphic User Interface (GUI), called Editable Hierarchy Knowledge (EHK), targeted specifically for complicated hierarchy tasks, as shown in Figure 1. The source code is released under the MIT License at <https://github.com/Teachable-AI-Lab/val>.

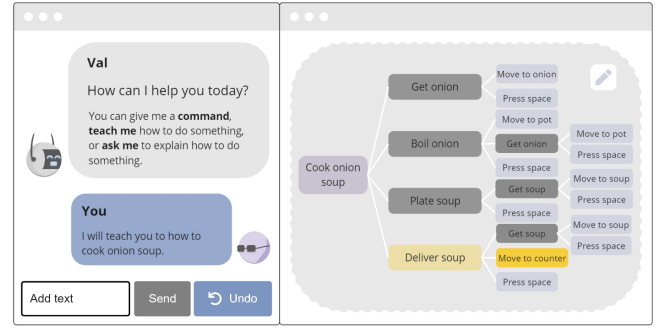


Figure 1: Example of Editable Hierarchy Knowledge GUI

First, we unified the language type. Particularly, we now describe the primitive tasks and known actions in natural language, reducing user's use of coding language that VAL cannot parse.

Second, we want VAL 2.0 to better fit different task instruction strategies. The proposed EHK is specially designed to extract and present the hierarchical structure from the user's instructions. It highlights the current step and folds previous subtasks in synchronous with the teaching process. The EHK interface provides granular feedback to the users that they are understood by the agent [1, 16]. If a flat hierarchy is detected, VAL will prompt a request for higher-level tasks, with EHK background flash as a visual alert.

Third, ongoing efforts are made to VAL 2.0 for better error handling by balancing between error prevention and recovery. To reduce the cumbersome confirmation dialogues, we added features to skip certain types of dialog. Instead, we offer several error recovery methods. VAL 2.0 allows users to revise the instruction directly via natural language in addition to undo. Moreover, in the EHK interface, users can delete an action or drag it to change the hierarchical structure of the task. We believe the EHK interface can be a way to compensate for the cost of reducing confirmation dialog frequency.

In the future, more tests will be done to test the usability of these new features, especially for the EHK interface. We believe the design insights and how we apply them to iterative design can inspire other ITL systems with complicated hierarchical tasks.

REFERENCES

- [1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. 2019. Guidelines for human-AI interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*. 1–13.
- [2] SSJA Baier and Sheila A McIlraith. 2009. HTN planning with preferences. In *21st Int. Joint Conf. on Artificial Intelligence*. 1790–1797.
- [3] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [4] Kevin A Gluck and John E Laird. 2019. *Interactive task learning: humans, robots, and agents acquiring new tasks through natural interactions*. Vol. 26. MIT Press.
- [5] Faria Huq, Jeffrey P Bigham, and Nikolas Martelaro. 2023. "What's important here?": Opportunities and Challenges of Using LLMs in Retrieving Information from Web Interfaces. *arXiv preprint arXiv:2312.06147* (2023).
- [6] James R Kirk, Robert E Wray, Peter Lindes, and John E Laird. 2024. Improving Knowledge Extraction from LLMs for Task Learning through Agent Analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18390–18398.
- [7] John E Laird, Kevin Gluck, John Anderson, Kenneth D Forbus, Odest Chadwicke Jenkins, Christian Lebiere, Dario Salvucci, Matthias Scheutz, Andrea Thomaz, Greg Trafton, et al. 2017. Interactive task learning. *IEEE Intelligent Systems* 32, 4 (2017), 6–21.
- [8] Lane Lawley and Christopher J. MacLellan. 2024. VAL: Interactive Task Learning with GPT Dialog Parsing. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. <https://doi.org/10.1145/3613904.3641915>
- [9] Amanda Li, Jason Wu, and Jeffrey P Bigham. 2023. Using LLMs to Customize the UI of Webpages. In *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–3.
- [10] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGILITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.
- [11] Toby Jia-Jun Li, Jingya Chen, Haijun Xia, Tom M Mitchell, and Brad A Myers. 2020. Multi-modal repairs of conversational breakdowns in task-oriented dialogs. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. 1094–1107.
- [12] Christopher J MacLellan, Erik Harpstead, Robert P Marinier III, and Kenneth R Koedinger. 2018. A framework for natural cognitive system training interactions. *Advances in Cognitive Systems* 6 (2018), 1–16.
- [13] Aaron Mininger and John E Laird. 2022. A Demonstration of Compositional, Hierarchical Interactive Task Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 13203–13205.
- [14] Don Norman. 2013. *The design of everyday things: Revised and expanded edition*. Basic books.
- [15] rabbit research team. 2023. Learning human actions on computer applications. <https://rabbit.tech/research>
- [16] Marcel Ruoff, Brad A Myers, and Alexander Maedche. 2023. ONYX: Assisting Users in Teaching Natural Language Interfaces Through Multi-Modal Interactive Task Learning. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [17] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandelkar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291 [cs.AI]*
- [18] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model based Autonomous Agents. *arXiv:2308.11432 [cs.AI]*
- [19] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents. *arXiv:2302.01560 [cs.AI]*